

Amlogic_NPU_LLM_User_ Guide

Version: v0.3

Release Date: 2026-08-24

版权

© 2026 晶晨半导体（上海）股份有限公司（代表其自身及其关联公司，以下简称“Amlogic”）版权所有。未经 Amlogic 书面许可，不得以任何形式或通过任何手段对本文件的任何部分进行复制、传播、转录或翻译成任何语言。

商标

 Amlogic，及 Amlogic 的其他图标均为 Amlogic 公司的商标。所有其他商标和注册商标均为其各自所有人的财产。

免责声明

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

本产品不适用于医疗、救生或生命维持等用途，也不适用于可合理预见因 Amlogic 产品故障或失灵可能导致人身伤害、死亡、严重财产或环境损害的应用场景。

文中提供的电路图及其他与 Amlogic 产品相关的信息仅为典型应用说明，完整且足以用于生产设计的全部信息不一定会提供。Amlogic 已尽量确保本文件内容准确和及时，但 Amlogic 不对本文档所含信息的准确性或完整性作任何保证。

Amlogic 产品可能存在未被识别的漏洞，或可能支持具有已知局限性的既定安全标准或规范。Amlogic 对任何漏洞不承担任何责任。

使用相关材料可能需获得第三方或 Amlogic 的知识产权许可。本文档对归属于 Amlogic 或任何其他方的知识产权不授予任何明示或暗示的声明或保证。文中包含的任何第三方网站链接仅为方便用户而提供，Amlogic 不对该等网站及其任何行为（包括隐私惯例、可用性和内容）作出认可，也不承担任何责任。

在任何情况下，Amlogic 均不对与本文档相关、由本文档使用或无法使用所产生的任何索赔承担责任。

联系方式

- 公司网址: www.amlogic.cn
- 售前咨询: contact@amlogic.com
- 技术支持: support@amlogic.com

修订记录

版本	说明
v0.1	初始版本
v0.2	去除对 Tokenizer 文件的依赖
v0.3(2026-08-24)	新增对 VLM 的支持

目录

1. Amlogic LLM 简介

- 1.1 LLMSDK 工作原理与流程
- 1.2 Amlogic LLM ToolChain介绍
- 1.3 适用的硬件平台和模型支持
- 1.4 关键字说明

2. 开发环境准备

- 2.1 开发环境确认
 - 2.1.1 平台类型确认
 - 2.1.2 DDK 版本确认
 - 2.1.3 系统环境确认
 - 2.1.4 设备DDR内存容量确认
 - 2.1.5 集成方式选择
- 2.2 Toolkit安装
 - 2.2.1 安装简介
 - 2.2.2 Toolkit安装
 - 2.2.2.1 配置虚拟环境
 - 2.2.2.2 安装amlInn_toolkit
 - 2.2.2.3 安装amllm_edge_toolkit_lite
 - 2.2.3 Native C 编译环境安装

3. 模型转换

- 3.1 转换简介
- 3.2 导出 GGUF 格式模型
 - 3.2.1 Hugging Face 模型下载
 - 3.2.2 llama.cpp安装
 - 3.2.3 huggingface转GGUF
- 3.3 GGUF模型转 ADLA
 - 3.3.1 转换示例
 - 3.3.1.1 LLM 转换示例
 - 3.3.1.2 VLM 转换示例
 - 3.3.2 接口说明
 - 3.3.2.1 AMLNN 初始化
 - 3.3.2.2 模型加载
 - 3.3.2.3 配置转换参数
 - 3.3.2.4 模型构建
 - 3.3.2.5 资源释放

4. 板端推理 (Python API)

- 4.1流程简介
- 4.2 Python Demo示例
 - 4.2.1 LLM 示例
 - 4.2.2 VLM 示例
- 4.3 Python API 接口说明
 - 4.3.1 初始化
 - 4.3.2 `setLogLevel`
 - 4.3.3 `config`
 - 4.3.4 `init`
 - 4.3.5 `run`
 - 4.3.6 `reset_session`
 - 4.3.7 `break_generation`
 - 4.3.8 `set_chat_template`
 - 4.3.9 `uninit`

5. 板端推理 (C++ API)

- 5.1 流程简介

5.2 基础数据结构和枚举

- 5.2.1 LLMContext
- 5.2.2 AML_LLMRunStatus
- 5.2.3 AML_LLMRetStatus
- 5.2.4 AML_LLMInputType
- 5.2.5 AML_LLMRunMode
- 5.2.6 AML_LLMImageType
- 5.2.7 AML_LLMSamplingMode
- 5.2.8 AML_LLMInitConfig
- 5.2.9 AML_LLMInitExtend
- 5.2.10 AML_LLMRunConfig
- 5.2.11 AML_LLMRunExtend
- 5.2.12 AML_LLMImageInput
- 5.2.13 AML_LLMMultimodalInput
- 5.2.14 AML_LLMTokenInput
- 5.2.15 AML_LLMInput
- 5.2.16 AML_LLMGenerationResult
- 5.2.17 AML_LLMEmbeddingResult
- 5.2.18 AML_LLMResult

5.3 基础 API 介绍

- 5.3.1 LLMResultCallback
- 5.3.2 aml_llm_init
- 5.3.3 aml_llm_run
- 5.3.4 aml_llm_uninit
- 5.3.5 aml_llm_reset
- 5.3.6 aml_llm_break
- 5.3.7 aml_llm_set_chat_template

5.4 错误返回码

5.5 多模态说明

- 5.5.1 多模态调用流程
- 5.5.2 VLM 示例

6. 常见问题与性能参考

- 6.1 性能参考数据
- 6.2 常见错误排查
- 6.3 参考链接

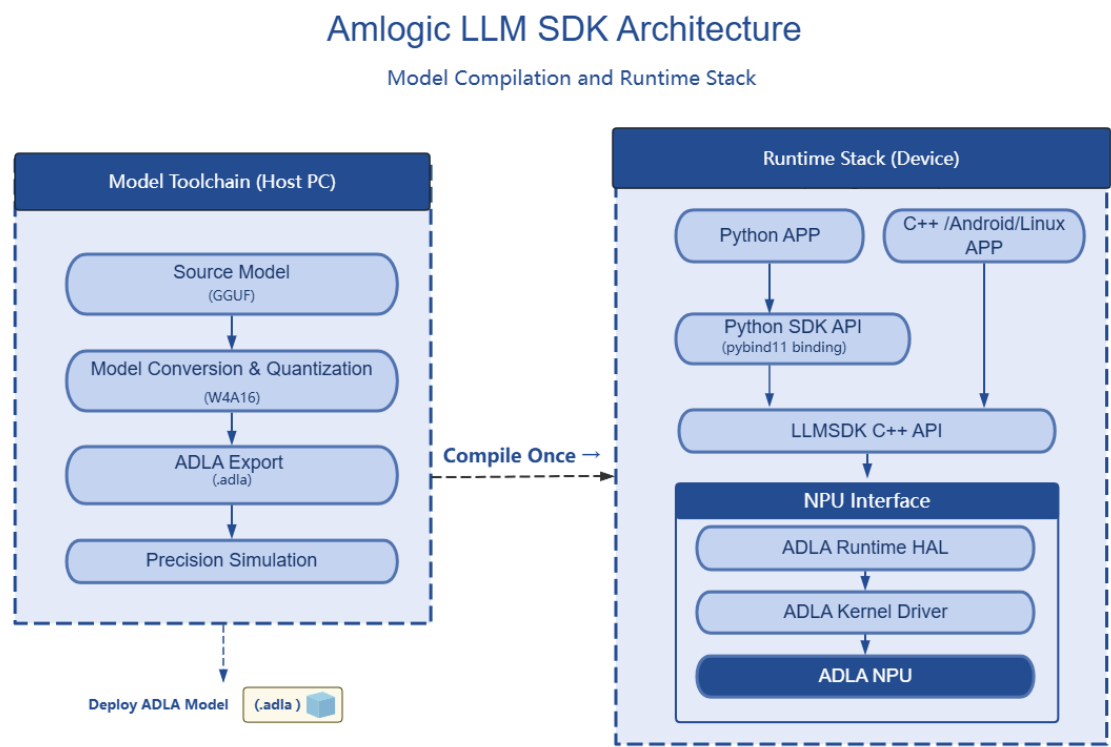
1. Amlogic LLM 简介

Amlogic LLM是面向大语言模型端侧部署的一站式软件解决方案，核心目标是降低主流大模型在Amlogic NPU平台的集成门槛，覆盖从模型转换到端侧推理的全流程，整体由[amlInn-toolkit](#)和[Runtime](#)两大核心组件构成：

- [amlInn-toolkit](#)：PC端模型转换工具包，支持LLM/VLM模型转换、量化、ADLA导出。
- **Runtime**：端侧高效推理执行层，封装底层NPU硬件细节，提供两级集成能力：
 - [Python Runtime \(amllm-edge-toolkit-lite\)](#)：轻量级Python包，接口极简，适合快速验证与应用原型开发。
 - [C++ Runtime \(libllmsdk.so\)](#)：面向Android/Linux的原生接口，支持深度系统集成与极致性能调优。

1.1 LLMSDK 工作原理与流程

软件栈架构图如下：



如图所示，LLMSDK 采用“一次编译+ 多端运行”的解耦架构，主要由 amlInn-toolkit 与 Runtime 两大核心组件构成：

- **开发转换侧**：用户输入GGUF等格式原始模型，通过amlInn-toolkit完成转换、量化、ADLA导出，生成面向Amlogic NPU深度优化的ADLA模型文件。
- **设备推理侧**：ADLA模型部署到Amlogic硬件后，Python/C++两条推理路径最终均通过NN HAL层统一调度，调用NPU硬件资源完成低功耗、高效率的端侧推理。

该架构大幅简化了大模型从云端到边缘设备的迁移路径，灵活适配不同开发场景的部署需求。

1.2 Amlogic LLM ToolChain介绍

ToolChain按「开发转换」和「板端推理」两个阶段拆分：

- **转换阶段 (x86 PC侧)** : `amlnn_toolkit` (X86架构whl包) , 仅支持f16/bf16/f32精度的GGUF格式LLM/VLM模型输入, 默认采用Q4A16量化方案 (Weight int4 group32, Activation f16) , 通过Python接口即可完成全流程量化与ADLA模型导出。
- **推理阶段 (板端Python)** : `aml11m_edge_toolkit_lite` (aarch64架构whl包) , 部署于边缘设备, 通过Python API直接加载已转换完成的ADLA模型, 完成LLM/VLM端侧推理执行。
- **推理阶段(11m_runtime库)**: 面向大语言模型场景的专属C++推理SDK, 链接 `lib11msdk.so`、引入 `11msdk.h` 头文件即可完成集成, 全量支持Android、Linux (Buildroot/Yocto/Debian) 平台, 提供模型加载、推理调度、上下文管理等LLM专属核心能力。

1.3 适用的硬件平台和模型支持

- **支持的硬件平台**
 - C305X2/A123X
 - A311Y3
- **已验证的支持模型列表**
 - 目前 SDK 已完成对以下主流开源模型的适配验证, 开发者可获取相应模型并进行端侧部署:

Category	Model series	Specific model
LLM	DeepSeek	DeepSeek-R1-Distill-Qwen-1.5B
LLM	Qwen	Qwen-1.8B-Chat ; Qwen1.5-0.5B-Chat ; Qwen1.5-1.8B-Chat ; Qwen2-0.5B-Instruct ; Qwen2-1.5B-Instruct ; Qwen2.5-0.5B-Instruct ; Qwen2.5-1.5B-Instruct ; Qwen2.5-3B-Instruct ; Qwen3-0.6B ; Qwen3-1.7B ; Qwen3-4B-instruct-2507
LLM	Gemma	gemma-2b-it ; gemma-2-2b-it ; gemma-3-270m-it ; gemma-3-1b-it ; gemma-4-E2B-it ; gemma-4-E4B-it
LLM	TinyLlama	TinyLlama-1.1B-Chat-v0.4 ; TinyLlama-1.1B-Chat-v1.0
LLM	Llama	Llama-3.1-8B-Instruct ; Llama-3.2-1B-Instruct ; Llama-3.2-3B-Instruct
LLM	Phi	phi-1.5 ; phi-2 ; Phi-3.5-mini-instruct
LLM	ChatGLM	chatglm3-6b
VLM	Qwen-VL	Qwen2-VL-2B-Instruct ; Qwen2.5-VL-3B-Instruct ; Qwen3-VL-2B-Instruct ; Qwen3-VL-4B-Instruct
VLM	InternVL	InternVL2-1B ; InternVL3-1B-Instruct
VLM	MiniCPM	MiniCPM3-4B ;

Note: `gemma-4-E2B-it` 与 `gemma-4-E4B-it` 模型目前仅支持通过 Amlogic 官方提供的预编译 ADLA 格式进行部署。当前版本的 `amlnn_toolkit` 暂不支持这两款模型的转换操作。

1.4 关键字说明

- **ADLA**: ADLA(amlogic Deep Learning Accelerator), 运行在amlogic 平台上的模型文件, 后缀为.adla。
- **Armbian (Debian)** : Debian系统, 后续文档中统一简称为 Armbian。

2. 开发环境准备

2.1 开发环境确认

2.1.1 平台类型确认

当前LLM方案仅支持A311Y3和C305X2/A123X两款Amlogic NPU平台，其余同系列平台暂未纳入支持范围。在启动模型转换流程前，必须先完成平台合规性校验，确认目标硬件处于官方支持范围内，从源头规避不兼容场景引发的后续部署故障。

在执行ADLA模型转换时，需获取与目标硬件完全匹配的 `target_platform` 参数，避免因参数 mismatch 导致转换失败或推理异常。平台类型校验可通过两种独立方式完成，所有校验结果均可对照《Part_number/Serial/target_platform对应关系表》，查询得到最终准确的 `target_platform` 取值。

Part_number	Serial	target_platform
A311Y3	Serial: 4d0a01020000000070e84a	PRODUCT_PID0XA007
C305X2/A123X	Serial: 4f0a01010000000054c6d1	PRODUCT_PID0XA008

方式一：通过 Part_number 快速确认

直接读取单板硬件表面刻印的Part_number标识，将读取到的编号与对应关系表匹配，即可获取该单板对应的正确 `target_platform` 参数。

方式二：通过 Serial 号校验确认

获取 Serial 信息：单板正常启动后，通过 ADB 连接设备，执行以下命令可读取 Serial 号：

```
adb shell cat /proc/cpuinfo
# 或
adb shell cat /proc/cpu_chipid
```

匹配规则：

- **首选规则：**优先截取Serial号的前4位字符，对照对应关系表完成匹配，获取目标 `target_platform` 值。
- **备选规则：**若前4位无匹配结果，可降级截取Serial号的前3位字符完成匹配，最终得到正确的 `target_platform` 参数。

2.1.2 DDK 版本确认

需要先确认NPU模块是否加载OK，并确认加载的 NPU Driver 版本

- **Driver加载确认：**adb shell 连接单板或直接基于单板命令窗口
 - **加载确认：**执行：lsmod |grep adla_core 结果中有 adla_core，如下图：

```
t7_an400:/ $ lsmod |grep adla_core
adla_core                221184  0
amlogic_cpufreq          16384  18 amwdec_av1_t5d_v41,amwdec_
```

- **权限确认：** 执行命令：ls /dev/adla0 -l 显示权限为666，如下图：

```
t7 an400:/ $ ls /dev/adla0 -la
crw-rw-rw- 1 system system 463,  0 1970-01-01 00:00 /dev/adla0
```

- **DDK版本确认：**
 - **Driver版本确认：** 重启单板，在串口或者在CMD窗口adb shell连接后，执行命令：dmesg |grep adla，获取到类似如下log：

```
5.405204] adla kmd version: 0.10.7.1
```

NPU Driver 版本为：0.10.7 (获取前三位对应的版本号即可)

- **C Runtime版本确认：** 板端支持如下确认命令

```
# Android 32 执行命令
strings /vendor/lib32/libllmsdk.so | grep "LLMSDK"

# Android 64 执行命令
strings /vendor/lib64/libllmsdk.so | grep "LLMSDK"

# Linux (Buildroot 32 / Buildroot 64 / Yocto 32 / Yocto 64 / Debian 64 /
Ubuntu 64) 执行命令
strings /usr/lib/libllmsdk.so | grep "LLMSDK"

# 预期输出示例：LLMSDK, v1.x.x, 202x.0x
```

- **DDK版本确认：**

需根据 NPU Driver 版本号确认对应的 DDK 版本，同时确保 C Runtime 及 amlnn-toolkit 版本匹配。

DDK版本	NPU Driver 版本	C Runtime Version	amlnn-toolkit version
DDK5.10.4.10	2.0.0		amlnn_toolkit_v1.x.x

Note：当前文档内容支持NPU Driver版本>=2.0.0以上的版本。

2.1.3 系统环境确认

本节用于确认板端系统环境与硬件架构，为后续章节选取对应版本的交叉编译工具链、确定集成方案提供准确依据。
若客户已明确当前使用的平台环境，可直接跳过本章节；若平台信息不确定，可参照以下步骤完成环境校验确认。

Step1 先确认是Android平台或者Linux平台

执行命令： adb shell getprop ro.product.cpu.abi

判定逻辑：

- **输出** `arm64-v8a` : 则确认为 64 位 Android 系统
- **输出** `armeabi-v7a` : 则确认为 32 位 Android 系统
- **报错** `command not found` : 判定为 Linux 系统, 请继续执行 Step 2

Step2 Linux 系统 (Buildroot / Yocto / Armbian) 环境进一步确认

Step2.1 确认是否为 Armbian / Yocto 系统

执行命令: `cat /version.txt`

参考输出示例(以 Armbian 为例)

```
image: a9-by401-bookworm-20260707030949
BRANCH=
VERSION=Linux/Debian/Armbian-PB#786
SPIN=Linux/Debian/Armbian-PB#786
BUILD_TIME="2026-07-07 03:27:41"
Generated on Tue Jul 7 03:27:41 UTC 2026

Compiler:
aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 11.3.Rel1) 11.3.1 20220712
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

判定逻辑:

- **有该文件存在:** 则判断为 armbian/Yocto 系统, 再通过 Compiler 版本信息判断是 32 位/64 位系统。参考示例对应 **Armbian 64-bit (AArch64)** 系统
- **文件不存在:** 命令执行失败, 判断为 Buildroot 系统, 请继续执行 Step 2.2

注意: 上述确认方式仅适用于 Amlogic 官方 Release 版本单板。第三方开发板请自行确认是否支持相同的版本文件与判定逻辑。

Step2.2 确认 Buildroot 系统架构

执行命令: `adb shell uname -m`

判定逻辑:

- `aarch64` : 则确认为 64 位 buildroot 系统
- `armv7l` : 则确认为 32 位 buildroot 系统

2.1.4 设备DDR内存容量确认

LLM 模型运行对可用 DDR 内存资源有较高要求, 需先确认设备实际 DDR 内存容量, 即可初步判定可承载的模型规模档位。不同参数量级的 LLM 模型, 对应设备侧需满足明确的 DDR 内存预留阈值, 保障 NPU 可顺利完成模型加载与稳定运行。

执行: `free -m`

```
akee:/data # free -m
              total        used          free      shared    buffers
Mem:           7925         1029          6895           3           0
-/+ buffers/cache:         1029          6895
Swap:           599          359           240
```

当前已支持模型的运行内存消耗可参考下表，该数值会随上下文长度(**Max Context**)、Compiler版本迭代产生变动，仅作为选型阶段的初步评估依据。

Model	Version	Param	Max Context	Memory (MB)
DeepSeek	R1	1.5B	2048	1268.62
Qwen	1	1.8B	2048	1634.78
Qwen	1.5	0.5B	2048	644.60
Qwen	1.5	1.8B	2048	1575.23
Qwen	2	0.5B	2048	402.92
Qwen	2	1.5B	2048	1075.24
Qwen	2.5	0.5B	2048	475.95
Qwen	2.5	1.5B	2048	1075.24
Qwen	2.5	3B	2048	2021.83
Qwen	3	0.6B	2048	779.52
Qwen	3	1.7B	2048	1541.24
Qwen	3	4B	2048	2842.98
Qwen	3	4B(Instruct)	2048	2842.98
TinyLlama	0.4	1.1B	2048	772.15
TinyLlama	1	1.1B	2048	772.06
Llama	2	7B	2048	5217.70
Llama	3.2	1B	2048	953.43
Llama	3.2	3B	2048	2347.69
Gemma	1	2B	2048	1836.43
Gemma	2	2B	2048	2130.33
Gemma	3	270M	2048	655.80
Gemma	3	1B	2048	1146.57
Gemma	4	E2B	2048	3542.01
Gemma	4	E4B	2048	5776.59
Phi	1.5	1.3B	2048	1404.81
Phi	2	2.7B	2048	2605.29
Phi	3	3.8B	2048	3400.62
ChatGLM3	3	6B	2048	4048.54

Model	Version	Param	Max Context	Memory (MB)
MiniCPM	3	4B	2048	4434.37
InternVL	2	1B	2048	526.52
InternVL	3	1B	2048	526.53
Qwen-VL	2	2B	2048	1315.36
Qwen-VL	2.5	3B	2048	2345.54
Qwen-VL	3	2B	2048	1704.13
Qwen-VL	3	4B	2048	3259.33

2.1.5 集成方式选择

- 支持的集成方式：

	Buildroot	Android	Debian	Yocto	Dependency
Native(c/c++ Api)	✓	✓	✓	✓	amlnn_toolkit /ADLA LLMSDK
Native(python Api)	×	×	✓	✓	amlnn_toolkit /amlllm_edge_toolkit_lite

- 集成方式介绍：

- Native(c/c++ Api)

- 使用toolkit将支持的LLM模型(gguf)转换成 ADLA 模型。
- Amlogic集成封装的统一LLMSDK接口，实现ADLA模型的加载和推理。

- Native (Python Api)

- 使用toolkit将支持的LLM模型(gguf)转换成 ADLA 模型。
- 再使用amlllm_edge_toolkit_lite的Python API实现ADLA模型的加载和推理。

- API 接口对比：

特性	Python Runtime (amlllm-edge-toolkit-lite)	C++ Runtime (libllmsdk.so)
语言特性	轻量级 Python 包，接口简洁	原生 C++ 接口，支持深度系统集成
性能/效率	性能良好，底层调用 C++ Runtime	性能极致，适用于性能敏感场景
开发速度	快速，便于原型开发和功能验证	相对较慢，集成和编译过程复杂
适用平台	Linux (aarch64, 如 Armbian/Yocto)	Android 和 Linux (全平台支持)
部署环境	Python 环境，依赖简单	依赖 NDK/交叉编译工具链和 libllmsdk.so

• **API选择建议:**

	适用用户/场景
Python Runtime	快速开发与验证型用户：追求最快的开发速度，需要快速验证模型效果；正在开发基于 Python 的应用、脚本或服务；对底层性能要求不是极致苛刻，仅在 Linux 设备上部署。
C++ Runtime	性能敏感和深度集成型用户：需要将 LLM 功能深度集成到 Android 或 Linux 原生系统应用中（如系统级服务、App）；追求最低延迟和最高推理性能；需要在 Android 平台或非 Python 环境中部署。

2.2 Toolkit安装

2.2.1 安装简介

不同单板系统环境支持不同的集成方式，不同集成环境需要安装的Tool chain不一样。先根据系统环境确定集成方式，再安装对应的Toolkit。

	amlInn_toolkit	amlllm_edge_toolkit_lite	Native C编译环境
Native(c/c++ Api)	✓	×	✓
Native(python Api)	✓	✓	×

2.2.2 Toolkit安装

选定集成方式后，如需安装 Toolkit，可参考本章安装说明。PC 端及 Debian 设备端均推荐使用 Miniforge 管理 Python 环境。

2.2.2.1 配置虚拟环境

- **安装包下载:**
 - x86_64: wget https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh
 - aarch64: wget <https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh>
- **Miniforge安装:**
 - chmod +x Miniforge3-Linux-*.sh
 - X86_64: bash Miniforge3-Linux-x86_64.sh
 - aarch64: bash Miniforge3-Linux-aarch64.sh
- **验证安装:**
 - conda config --show channels 执行成功，结果中有 - conda-forge
- **虚拟环境创建**
 - 创建虚拟环境: conda create -n amlInn_toolkit python=3.10
 - 进入虚拟环境: conda activate amlInn_toolkit
 - Python 版本可按需调整（推荐 3.10）、'amlInn_toolkit' 为虚拟环境名称，可自行修改

2.2.2.2 安装amlnn_toolkit

`amlnn_toolkit` 为PC端安装的模型转换工具包，需先激活对应的Conda环境，再通过本地wheel包完成安装；若直接使用预编译ADLA模型（参考3.1章节说明），可直接跳过该工具的安装步骤。

Install:

根据 Python 版本及处理器架构，选择对应的 `requirements` 和 `whl` 文件。

download link: https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/whl/linux_x86

#for amlnn_toolkit

Step1:逐条安装依赖（避免版本冲突）

```
for req in $(cat requirements_x86_64.txt); do pip install $req; done
```

Step2:安装 toolkit whl 包

```
# toolkit-x.x.x 为工具包版本号, cpxx 为 Python 版本标签（如 cp310、cp311）
pip install amlnn_toolkit-x.x.x-cpxxx-cpxxx-linux_x86_64.whl
```

#Note: 请勿使用 `pip install -r requirements_x86_64.txt`，会导致依赖版本冲突。必须逐条安装。

Verify:

#验证toolkit-x.x.x版本

```
python -c "from amlnn.api import AMLNN_LLM; print('amlnn_toolkit import OK')"
```

2.2.2.3 安装amllm_edge_toolkit_lite

在激活 `amlnn_toolkit` Conda 环境后，请通过本地 wheel 包完成安装。需注意，

`amllm_edge_toolkit_lite` 是专为板端 Yocto/Armbian 系统环境设计的模型推理工具包，用于边缘侧部署。

Install:

download link: https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit_lite/amllm_edge_toolkit_lite/whl/aarch64

#for amllm_edge_toolkit_lite

```
pip install -r requirements.txt
```

```
pip install amllm_edge_toolkit_lite-1.x.x-cp310-cp310-linux_aarch64.whl
```

Verify:

#验证toolkit_lite-x.x.x版本

```
python -c "from amllmlite.api import AMLLLMLite; print('amllm_edge_toolkit_lite import OK')"
```

2.2.3 Native C 编译环境安装

Native c编译环境安装是在PC端安装编译设备系统对应的交叉编译工具。不同单板系统环境需要安装不同的工具链，具体如下：

- **Android System:**

- 交叉编译工具下载链接：[Andoird NDK下载地址](#)，推荐使用 android-ndk-r25c-linux.zip。
- 解压到指定目录,如：/opt/android-ndk-r25c
- 编译配置
 - 编译时，在 `build-android64.sh` / `build-android32.sh` 中将 `ANDROID_NDK_PATH` 指定为本地 toolchain 路径。

- **Buildroot System:**

- 交叉编译工具下载链接：
 - 32bit: [Buildroot 32 toolchain下载地址](#)
 - 64bit: [Buildroot 64 toolchain下载地址](#)
- 解压到指定目录，如：
 - 32bit: /opt/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi
 - 64bit: /opt/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu
- 编译配置
 - 编译时，在 `build-buildroot64.sh` / `build-buildroot32.sh` 中将 `BUILDROOT_TOOLCHAIN_PATH` 指定为本地 toolchain 路径。

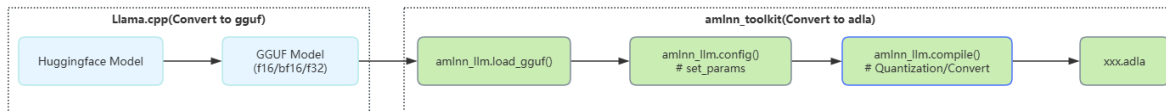
- **Debian/Yocto System:**

- 交叉编译工具下载链接：
 - 32bit: [32 toolchain下载地址](#)
 - 64bit: [64 toolchain下载地址](#)
- 执行对应脚本：
 - 32bit: `./poky-glibc-x86_64-amlogic-bsp-armv7at2hf-neon-mesons7-bh201-5.15-a32-toolchain-4.0.20.sh`
 - 64bit: `./poky-glibc-x86_64-meta-toolchain-armv8a-mesont7-an400-5.15-a64-toolchain-4.0.20.sh`
- 安装选择：
 - 选择安装路径，如: /opt/poky/4.0.20。
 - 选择输入 yes。
 - 稍等片刻后，编译工具链即可安装完毕。
- 编译配置
 - 编译时，在 `build-yocto64.sh` / `build-yocto32.sh` 中将 `YOCTO_TOOLCHAIN_PATH` 指定为本地 toolchain 路径。

3. 模型转换

`amlnn-toolkit` 提供模型转换功能，支持将 GGUF 格式的大语言模型，转换为可在 Amlogic NPU 上运行的 ADLA 格式模型。本节将详细说明 `amlnn-toolkit` 针对 LLM 模型的完整转换流程，为用户提供操作参考。

3.1 转换简介



当前 `amlnn-toolkit` 仅支持 **GGUF** 格式的 LLM 模型的导入与转换。因此，在启动转换流程前，请务必确保已获取目标模型的 GGUF 格式文件。上图中的预处理部分（即图中“Convert to GGUF”环节）需由用户自行完成。可通过以下三种方式获取最终的 ADLA 模型：

1. 使用预编译 ADLA 模型：

- 直接从 [Amlogic-NN](#) 官方 Hugging Face 仓库下载预编译完成的 ADLA 模型文件，可直接导入边缘板端运行，大幅缩减部署准备周期。

2. 下载 GGUF 并转换生成 ADLA：

- 从 Hugging Face 等开源模型社区直接下载官方已发布的标准 GGUF 格式模型文件
- 导入 `amlnn-toolkit` 工具进行 ADLA 转换

3. 自行转换生成 GGUF 后转 ADLA：

- 使用 `llama.cpp` 官方提供的开源转换工具，将原始 Hugging Face 格式的原生模型转换为标准 GGUF 格式模型文件。
- 导入 `amlnn-toolkit` 工具进行 ADLA 转换

3.2 导出 GGUF 格式模型

GGUF 是由 `llama.cpp` 社区主导推出的高效二进制模型格式，专为大语言模型（LLM）在端侧及本地环境的高性能推理而设计。

- 若已具备 GGUF 模型：**如果您已从 Hugging Face 等社区直接下载了目标模型的 GGUF 格式文件，请**跳过本章**，直接进入后续模型转换或部署环节。
- 若需自行转换：**本章将详细讲解如何利用 `llama.cpp` 提供的官方工具，将原始的 Hugging Face 格式模型转换为标准的 GGUF 格式。

3.2.1 Hugging Face 模型下载

确保已下载完整的 HF 模型文件（包含 `.safetensors` 或 `.bin`、`tokenizer.json` 等）。

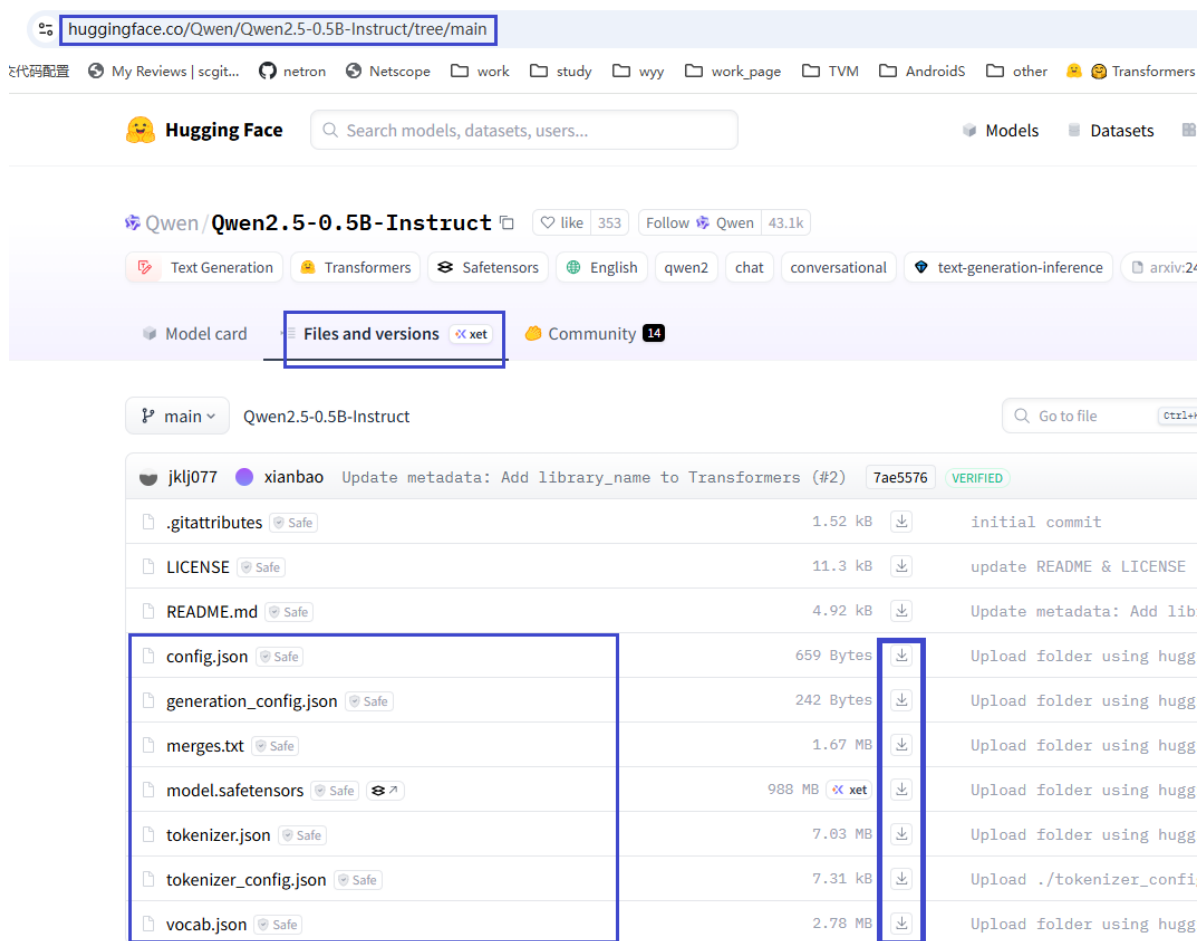
方式一：命令行下载

```
pip install huggingface_hub
pip install -U huggingface_hub[cli]
# Optional: Login to access gated model
huggingface-cli login
# Download the model (disable symlinks for compatibility)
huggingface-cli download Qwen/Qwen2.5-0.5B \
    --local-dir ./qwen2.5-0.5b --local-dir-use-symlinks False
```

Note: 用该方式下载可能会因为网络原因下载模型失败, 此时建议使用方式二。

方式二: 从网页下载

打开支持模型列表中的 link, 点击 **Files and versions**, 然后可以直接点击文件右边的下载链接下载原始模型文件, 如下图:



一般 Hugging Face 转 GGUF, 只需要下载 `config.json`、`safetensors` (或 `pytorch_model.bin`)、`tokenizer.json` (或 `tokenizer.model` 或 `xx.tiktoken`) 即可。

3.2.2 llama.cpp安装

将huggingface转gguf是通过llama.cpp完成, 当前需先下载 llama.cpp 源码并安装依赖, 命令如下:

```
conda create -n llama_cpp python=3.10
conda activate llama_cpp

git clone --depth 1 --branch b6199 https://github.com/ggerganov/llama.cpp.git
cd llama.cpp
pip install -r requirements.txt
```

3.2.3 huggingface转GGUF

使用 llama.cpp 提供的 `convert_hf_to_gguf.py` 脚本, 可将 Hugging Face 格式的模型转换为 GGUF 格式。以下以 Qwen2.5-0.5B 模型为例进行说明:

```
# usage: python convert_hf_to_gguf.py [model_dir] --outfile [out_path]
python convert_hf_to_gguf.py \
    ./qwen2.5-0.5b \
    --outfile qwen2.5-0.5b_fp16.gguf \
    --outtype f16
```

参数说明：

参数/路径	说明
./qwen2.5-0.5b	输入路径：Qwen2.5-0.5B Hugging Face 模型的本地文件夹路径。
--outfile	输出文件：指定生成的 GGUF 模型文件名（如 qwen2.5-0.5b_fp16.gguf）。
--outtype	精度类型：指定输出模型的权重精度，f16 表示半精度浮点数。

执行结果：

命令执行成功后，将在当前目录下生成名为 `qwen2.5-0.5b_fp16.gguf` 的模型文件，该文件即可用于后续的 `amlnn-toolkit` 转换流程。

3.3 GGUF模型转 ADLA

3.3.1 转换示例

转换过程通过 `amlnn-toolkit` 包提供的接口完成，主要包含加载、构建、导出三个步骤。

3.3.1.1 LLM 转换示例

```
from amlnn.api import AMLNN_LLM

# AMLNN_LLM init
amlnn_llm = AMLNN_LLM(log_level='INFO')

modelpath="./Qwen2.5-0.5B-Instruct-F16.gguf"

# load gguf model
amlnn_llm.load_gguf(model=modelpath)

amlnn_llm.config(prefill_batch_length=32, max_context=2048 ,
    transformer_model_type='LLM',target_platform='PRODUCT_PID0XA007',
    output_dir="output")

amlnn_llm.compile()

amlnn_llm.uninit()
```

3.3.1.2 VLM 转换示例

```
from amlnn.api import AMLNN_LLM

# AMLNN_LLM init
amlnn_llm = AMLNN_LLM(log_level='INFO')
```

```
#for VLM's vision model
modelpath="./mmproj-F32.gguf"
amlnn_llm.load_gguf(model=modelpath)
amlnn_llm.config(transformer_model_type='MMPROJ',target_platform='PRODUCT_PID0XA007', output_dir="output")
amlnn_llm.compile()
amlnn_llm.uninit()

#for VLM's LLM model
modelpath="./Qwen3-VL-4B-Instruct-BF16.gguf"
amlnn_llm.load_gguf(model=modelpath)
amlnn_llm.config(prefill_batch_length=32,
max_context=2048,transformer_model_type='VLM',target_platform='PRODUCT_PID0XA007', output_dir="output")
amlnn_llm.compile()
amlnn_llm.uninit()
```

3.3.2 接口说明

3.3.2.1 AMLNN 初始化

在使用Amlnn-toolkit的API接口时，需要先调用AMLNN_LLM()进行AMLNN_LLM对象初始化，结束时调用uninit()方法进行释放。

参数	类型	说明	必要参数
log_level	str	log 等级设置， 默认值： 'INFO' 。当前支持['DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL']	否

```
from amlnn.api import AMLNN_LLM
amlnn_llm = AMLNN_LLM(log_level='INFO')

...

amlnn_llm.uninit()
```

3.3.2.2 模型加载

功能：加载gguf模型

返回值：0：成功 ， -1：失败

参数	类型	说明	必要参数
model	str	gguf模型文件路径	是

函数定义如下：

```
#example
modelpath="./Qwen2.5-0.5B-Instruct-F16.gguf"

amlnn_llm.load_gguf(model=modelpath)
```

• **VLM 视觉编码器特殊转换流程说明**

当 VLM 模型的 Vision 模块（视觉编码器）无法兼容 GGUF 格式时，禁止直接使用 GGUF 转换链路，需切换至 ONNX 转换通道完成 ADLA 格式输出。

- **操作指引：**
 - **适用场景：**仅针对 Vision Encoder 无法通过标准 GGUF 路径完成转换的场景，典型示例：**FastVLM 0.5B 视觉模块**。
 - **参考文档：**遵循《03_Amlogic_NPU_API_Reference_AMLNN》中 ONNX 模型转换流程
 - **完整转换链路：**Vision 模块源模型 --> 导出标准 ONNX 格式 -->ADLA

3.3.2.3 配置转换参数

功能： 配置模型转换参数。

返回值： 0：成功 ， -1：失败

参数	类型	说明	必要参数
prefill_batch_length	int	控制单次prefill阶段一次性处理的最大token数量。 推荐值 [32,64]。默认值:32。 transformer_model_type为LLM/VLM 时有效。	否
max_context	int	最大上下文，推荐值 [256,512,1024,2048,4096,8192,16384]。默认值： 2048。 transformer_model_type为LLM/VLM 时有效。	否
transformer_model_type	str	有效值:['MMPROJ', 'LLM', 'VLM']。 默认值：LLM 。 若仅部署纯文本大模型，保持默认值 LLM 即可。若 部署多模态大模型（VLM），需分两步转换：转换视 觉部分时设置 MMPROJ ，转换语言部分设置 VLM 。	是
target_platform	str	platform type，支持 PRODUCT_PID0XA007 、 PRODUCT_PID0XA008 。默认值： PRODUCT_PID0XA007 根据2.1.1章节：平台类型确认，获取正确的值。	是
output_dir	str	输出路径， 默认值：None ，表示默认输出路径为： model_name + "_adla_unify"	否

```
#Example for LLM
amlnn_llm.config(prefill_batch_length=32, max_context=2048 ,
transformer_model_type='LLM',target_platform='PRODUCT_PID0XA007',
output_dir="output")

#Example for VLM's Vision
amlnn_llm.config(transformer_model_type='MMPROJ',target_platform='PRODUCT_PID0XA007', output_dir="output")

#Example for VLM's LLM
amlnn_llm.config(prefill_batch_length=32, max_context=2048 ,
transformer_model_type='VLM',target_platform='PRODUCT_PID0XA007',
output_dir="output")
```

3.3.2.4 模型构建

功能： 编译AMLLLM模型，并输出最终的adla文件。

返回值： 0：成功， -1：失败

```
#example
amlnn_llm.compile()
```

3.3.2.5 资源释放

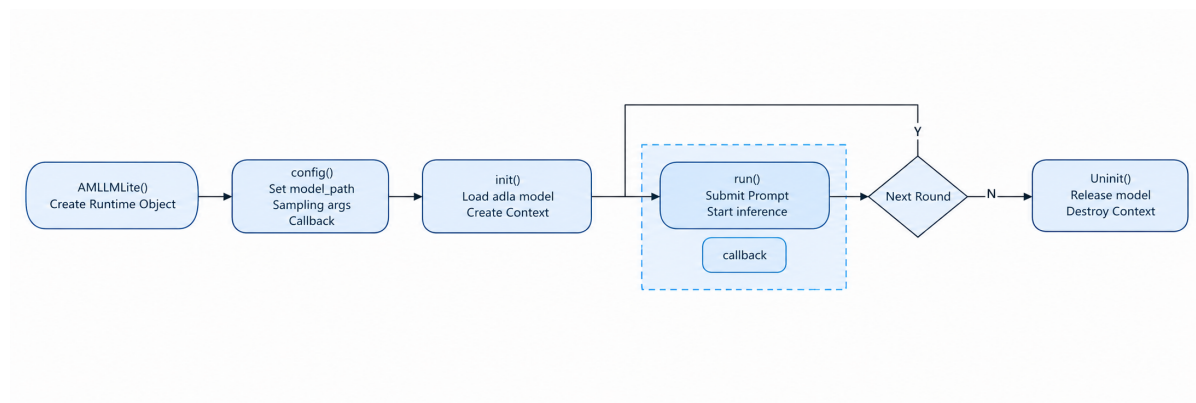
功能： 释放运行时与模型资源

```
#Example
amlnn_llm.uninit()
```

4. 板端推理 (Python API)

本章详细介绍 Amlogic LLMSDK 中的 Python Runtime 组件 `amllm-edge-toolkit-lite`，以及它在边缘板端的完整推理执行流程。这套 Python API 提供了轻量化的开发接口，可用于快速搭建原型与功能验证，底层会自动调用高性能 C++ Runtime 调度 NPU 完成全硬件加速推理。

4.1 流程简介



Python API 全链路调用流程如上图所示。开发者仅需导入 `amllmlite` 包中的 `AMLLMLite` 类，即可快速完成边缘端LLM/VLM应用的二次开发，完整操作步骤如下：**初始化Runtime对象 --> 加载语言模型 --> 加载VLM的视觉模型(VLM专属) --> 传入推理请求 --> 流式输出与结果汇总 --> 统一释放底层资源**

- **环境准备：** 参考 2.2.2.3 安装`amllm-edge-toolkit-lite`章节，在板端环境下安装好工具推理包。
- **核心函数简要说明**
 - **实例化 (`AMLLMLite()`)**
 - 原理：在 Python 进程中创建一个 Runtime 对象，`log_level` 可设置为 `DEBUG`、`INFO`、`WARNING` 或 `ERROR`，用于控制 Python 层日志输出。此时只创建对象，不加载模型。
 - **参数配置 (`llm.config(...)`)**
 - 原理：Python 层收集用户提供的模型路径、可选 `mmproj_path`、采样参数和 Token 回调，将这些信息存储起来，等待初始化时传递给底层 C++ Runtime 结构体（如 `AML_LLMLiteInitConfig`）。其中 `model_path` 必须指向板端可访问的语言模型 `.adla` 文件；VLM 还必须提供对应的 `mmproj_path`。`sampling_mode` 可选择 `argmax`、`top_p` 或 `top_k`，并按需设置 `top_k`、`top_p`、`temperature` 和 `repeat_penalty`；`on_token` 用于接收流式生成结果。
 - **初始化 (`llm.init()`)**
 - 原理：这是资源分配的关键步骤。Python Wrapper 调用 C++ Runtime 的 `aml_llm_init()` 接口。`init()` 不接收模型路径或采样参数，这些参数必须提前通过 `config()` 配置完成；初始化成功后才能调用 `run()`。
 - 底层执行：
 - 模型加载：C++ Runtime 从磁盘加载 `.adla` 模型文件，进行完整性校验。
 - NPU 上下文创建：向 NPU 驱动申请硬件资源，如分配 NPU 专用的 KV Cache 内存（用于存储历史 Key/Value 状态）和模型权重内存。
 - 状态就绪：完成硬件抽象层初始化，LLM 实例处于可接受推理请求的状态。
 - **执行推理 (`llm.run(...)`)**

- 原理：启动模型的核心计算流程。Python 层将用户的 Prompt 字符串以及可选的多模态图片输入传入 C++ Runtime。prompt 是当前轮文本输入，input_type 使用 prompt 表示文本请求，使用 multimodal 表示图文请求；VLM 请求还需在 images 中提供包含 data、width 和 height 的 RGB888 图像，并保证 img_content 与 Prompt 中的图像标记一致。
- 底层执行：
 - 编码与预处理：C++ Runtime 调用内置 Tokenizer，将输入 Prompt 文本（UTF-8）转换为 Token ID 序列；VLM 输入还会处理 RGB888 图片及 projector 模型。
 - Kernel 调度：将 Token ID 序列及必要的图像特征作为模型输入，调度 NPU Kernel 进行前向计算。
 - 流式输出：每个 Token 生成后，C++ Runtime 触发 Callback。Python Runtime 捕获并处理这些回调，实现用户友好的流式输出，并在 run() 返回时聚合最终结果。
- 资源释放 (llm.uninit())
 - 原理：销毁 LLM 实例，清理 NPU 占用的内存。应用应在所有 run() 请求和回调处理完成后调用 uninit()；该接口不需要额外参数，重复调用或未完成初始化时也应安全处理。
 - 底层执行：调用 C++ Runtime 的 aml_llm_uninit() 接口，释放模型占用的 KV Cache 和权重内存，并销毁 NPU 上下文句柄。

4.2 Python Demo示例

4.2.1 LLM 示例

调用顺序：创建对象、配置参数、初始化、推理，释放。回调在每个 Token 到达时输出文本。

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""Interactive AMLLLMLite demo that mirrors the C sample."""

import argparse
# argparse 用于解析模型路径和采样参数。
import queue
# 队列把输入线程收到的请求安全地交给推理线程。
import sys
import threading
from amllmlite.api import AMLLLMLite
from amllmlite.api.inference import RunStatus

def stream_callback(token, userdata=None):
    # 回调在每个 Token 到达时触发，FINISH/ERROR 表示本次生成结束或失败。
    """Print tokens as they arrive (mimic C demo callback behavior)."""
    text = token.get("text", "")
    status = token.get("status")
    if userdata and not userdata.get("printed"):
        print(f"[Request #{userdata.get('request_id', 0)}]")
        userdata["printed"] = True
    if status == RunStatus.FINISH:
        print()
    elif status == RunStatus.ERROR:
```

```

        print("\n[Generation error]")
    elif text:
        print(text, end="", flush=True)

def parse_args():
    # 命令行参数与 AMLLLMLite.config() 的配置项一一对应。
    parser = argparse.ArgumentParser(description="Amlogic LLM interactive demo (Python)")
    parser.add_argument("--model", required=True, help="Path to LLM model file")
    parser.add_argument("--sampling-mode", default="argmax", choices=["argmax", "top_p", "top_k"], help="Sampling mode")
    parser.add_argument("--top-k", type=int, default=3, dest="top_k", help="Top-K parameter")
    parser.add_argument("--top-p", type=float, default=0.9, dest="top_p", help="Top-P parameter")
    parser.add_argument("--temperature", type=float, default=1.0, help="Softmax temperature")
    parser.add_argument("--repeat-penalty", type=float, default=1.1, dest="repeat_penalty", help="Repeat penalty factor")
    parser.add_argument("--log-level", default="ERROR", dest="log_level", choices=["DEBUG", "INFO", "WARNING", "ERROR"])
    return parser.parse_args()

def main():
    # 标准生命周期: 创建、config、init、循环 run、finally 中 uninit。
    args = parse_args()
    amlllm = AMLLLMLite(log_level=args.log_level)
    # config 必须在 init 前调用; 模型路径必须指向板端可访问的 ADLA 文件。
    amlllm.config(
        model_path=args.model,
        sampling_mode=args.sampling_mode,
        top_k=args.top_k,
        top_p=args.top_p,
        temperature=args.temperature,
        repeat_penalty=args.repeat_penalty,
        on_token=stream_callback,
    )
    amlllm.init()

    print("Welcome to Amlogic LLM interactive demo (Python).")
    print("Commands: exit | new_talk | break")

    user_state = {"request_id": 0, "printed": False}

    input_queue = queue.Queue()

    def read_input():
        while True:
            try:
                user_input = input().strip()
            except EOFError:
                input_queue.put("exit")
                break

```

```

if user_input == "break":
    try:
        amlllm.break_generation()
        print("\nStop signal sent.")
    except Exception as exc:
        print(f"\nBreak failed: {exc}")
    continue
if user_input == "new_talk":
    try:
        amlllm.reset_session()
        print("\nConversation state cleared.")
    except Exception as exc:
        print(f"\nReset failed: {exc}")
    continue
input_queue.put(user_input)
if user_input == "exit":
    break

```

```

input_thread = threading.Thread(target=read_input, daemon=True)
input_thread.start()

```

```

try:
    while True:
        print("\nLLM@Amlogic>>> ", end="", flush=True)
        user_input = input_queue.get()
        if not user_input:
            print("Please enter a non-empty prompt.")
            continue
        if user_input == "exit":
            break

        try:
            user_state["request_id"] += 1
            user_state["printed"] = False
            # prompt 模式不需要图片; run 返回聚合文本和 Token 统计。
            result = amlllm.run(
                prompt=user_input,
                input_type="prompt",
                run_mode="generate",
                retain_history=False,
                enable_think=False,
                role="user",
                user_data=user_state,
            )
            if not result["text"].endswith("\n"):
                print()
            print(f"Tokens generated: {result['token_count']}")
        except KeyboardInterrupt:
            print("\nKeyboardInterrupt received. Sending break...")
            amlllm.break_generation()
        except Exception as exc:
            print(f"\nGeneration failed: {exc}")
finally:
    amlllm.uninit()

```

```
if __name__ == "__main__":
    sys.exit(main())
```

`retain_history` 在该示例中显式设置为 `False`。需要多轮历史时，将其改为 `True`，或在应用代码中按请求传入；输入 `new_talk` 可清空会话，输入 `break` 可中断当前生成。

4.2.2 VLM 示例

调用顺序：创建对象、配置参数、初始化、推理，释放。VLM配置时增加 `mmproj_path`，运行时传入 RGB888 图片。Pillow 只负责把图片文件解码为紧密排列的 RGB888 字节，不属于 Runtime 必需依赖。

```
#!/usr/bin/env python3
"""Interactive AMLLLMLite VLM demo corresponding to LLM SDK main_vlm.cpp."""

import argparse
# argparse 用于解析模型路径和采样参数。
import sys

from amllmlite.api import AMLLLMLite
from amllmlite.api.inference import RunStatus

def stream_callback(token):
    status = token.get("status")
    if status == RunStatus.FINISH:
        print()
    elif status == RunStatus.ERROR:
        print("\n[Generation error]")
    elif token.get("text"):
        print(token["text"], end="", flush=True)

def load_rgb_image(path):
    # 将任意 Pillow 支持的格式转换成 RGB 紧密排列的 data。
    try:
        from PIL import Image
        # Pillow 负责图片解码，Runtime 本身只接收 RGB888 字节。
    except ImportError as exc:
        raise RuntimeError("VLM demo requires Pillow: pip install Pillow") from exc

    with Image.open(path) as image:
        rgb = image.convert("RGB")
        width, height = rgb.size
        return {"data": rgb.tobytes(), "width": width, "height": height}

def parse_args():
    # 命令行参数与 AMLLLMLite.config() 的配置项一一对应。
    parser = argparse.ArgumentParser(description="Amlogic VLM interactive demo")
    parser.add_argument("--model", required=True, help="Path to LLM model file")
```

```

    parser.add_argument("--mmproj", required=True, help="Path to multimodal projector model file")
    parser.add_argument("--sampling-mode", default="argmax", choices=["argmax", "top_p", "top_k"])
    parser.add_argument("--top-k", type=int, default=3, dest="top_k", help="Top-K parameter")
    parser.add_argument("--top-p", type=float, default=0.9, dest="top_p", help="Top-P parameter")
    parser.add_argument("--temperature", type=float, default=1.0, help="Softmax temperature")
    parser.add_argument("--repeat-penalty", type=float, default=1.1, dest="repeat_penalty", help="Repeat penalty factor")
    parser.add_argument("--log-level", default="ERROR", dest="log_level", choices=["DEBUG", "INFO", "WARNING", "ERROR"])
    parser.add_argument("--img-start", default="<|vision_start|>")
    parser.add_argument("--img-end", default="<|vision_end|>")
    parser.add_argument("--img-content", default="<image>")
    return parser.parse_args()

```

```

def main():
    # 标准生命周期: 创建、config、init、循环 run、finally 中 uninit。
    args = parse_args()
    amlllm = AMLLLMLite(log_level=args.log_level)
    # config 必须在 init 前调用; 模型路径必须指向板端可访问的 ADLA 文件。
    amlllm.config(
        model_path=args.model,
        mmproj_path=args.mmproj,
        sampling_mode=args.sampling_mode,
        top_k=args.top_k,
        top_p=args.top_p,
        temperature=args.temperature,
        repeat_penalty=args.repeat_penalty,
        on_token=stream_callback,
    )
    amlllm.init()
    # images 保存待下一轮使用的图片; 推理完成后会清空。
    images = []

    print("VLM model loaded. Use /image <path> to append images, then enter a prompt.")
    print("Commands: exit | new_talk | break")
    try:
        while True:
            try:
                line = input("\nVLM>>> ").strip()
            except EOFError:
                break
            if not line:
                print("Please enter a non-empty prompt.")
                continue
            if line == "exit":
                break
            if line == "new_talk":
                amlllm.reset_session()

```

```

        images.clear()
        print("Conversation state cleared.")
        continue
    if line == "break":
        amlllm.break_generation()
        print("Stop signal sent.")
        continue
    # /image 只加载并暂存图片，不立即触发推理。
    if line.startswith("/image "):
        path = line[7:].strip()
        try:
            image = load_rgb_image(path)
        except Exception as exc:
            print(f"Image load failed: {exc}")
            continue
        images.append(image)
        print(
            f"Image appended: {path},
{image['width']}x{image['height']}, "
            f"total={len(images)}"
        )
        continue

    prompt = line
    if images and args.img_content not in prompt:
        prompt = args.img_content * len(images) + prompt
        print(f"Prepended {len(images)} image marker(s).")

    print("robot: ", end="", flush=True)
    # multimodal 模式要求 mmproj_path、images 和图片标记同时有效。
    result = amlllm.run(
        prompt=prompt,
        input_type="multimodal" if images else "prompt",
        retain_history=False,
        enable_think=False,
        role="user",
        images=images or None,
        img_start=args.img_start,
        img_end=args.img_end,
        img_content=args.img_content,
    )
    if not result["text"].endswith("\n"):
        print()
    images.clear()
finally:
    amlllm.uninit()
return 0

if __name__ == "__main__":
    sys.exit(main())

```

输入 `/image <path>` 可暂存一张或多张图片；下一条普通文本会使用这些图片并在推理后清空暂存列表。`new_talk` 同时清空 Runtime 会话和图片列表。若 Prompt 中没有 `<image>`，示例会自动在开头补齐与图片数量相同的标记。启用跨轮历史时，应将两个 demo 中的 `retain_history=False` 改为 `True`。

4.3 Python API 接口说明

以下接口均属于 `amllmlite.api.AMLLMLite`。除 `__init__` 外，接口应按“`config()` → `init()` → `run()` → `uninit()`”顺序使用；未满足前置条件时 Runtime 会抛出异常。

4.3.1 初始化

功能：创建 Python Runtime 对象，设置 Python 侧日志等级；此时尚未加载模型。

参数	类型	说明
<code>log_level</code>	<code>str</code>	日志等级，可选 <code>DEBUG</code> 、 <code>INFO</code> 、 <code>WARNING</code> 、 <code>ERROR</code> 。

返回值：`AMLLMLite` 实例。

```
from amlmlite.api import AMLLLMLite

amllm = AMLLLMLite(log_level="DEBUG")
```

4.3.2 `setloglevel`

功能：运行过程中更新日志等级。

参数	类型	说明
<code>level</code>	<code>str</code>	可选 <code>DEBUG</code> 、 <code>INFO</code> 、 <code>WARNING</code> 、 <code>ERROR</code> 。

返回值：无。

4.3.3 `config`

功能：保存模型路径、VLM projector、采样策略和 Token 回调，创建底层 Runtime；必须在 `init()` 前调用。重复调用会关闭并重建已有 Runtime。

参数	类型	说明
<code>model_path</code>	<code>str</code>	LLM 或 VLM 语言模型 <code>.ad1a</code> 文件，路径必须存在。
<code>mmproj_path</code>	<code>str / None</code>	多模态 projector <code>.ad1a</code> 文件；纯 LLM 留空。
<code>sampling_mode</code>	<code>str</code>	<code>argmax</code> 、 <code>top_p</code> 、 <code>top_k</code> （兼容别名 <code>topk</code> ）。
<code>top_k</code>	<code>int</code>	Top-K 候选数，必须大于 0。
<code>top_p</code>	<code>float</code>	Top-P 累积概率，范围 <code>(0, 1]</code> 。

参数	类型	说明
<code>temperature</code>	<code>float</code>	Softmax 温度，必须大于 0。
<code>repeat_penalty</code>	<code>float</code>	重复惩罚系数，必须大于 0。
<code>on_token</code>	<code>callable / None</code>	每个 Token 触发一次的回调，签名可为 <code>(token)</code> 或 <code>(token, user_data)</code> 。

返回值：无。路径、采样参数或回调不合法时抛出 `ValueError`、`FileNotFoundError`。

4.3.4 `init`

功能：加载 ADLA 模型、创建 NPU 上下文并分配推理所需资源。

返回值：无；初始化失败抛出 `RuntimeError` 或底层异常。

4.3.5 `run`

功能：执行一次文本或多模态生成，并通过 `on_token` 流式返回结果。

参数	类型	说明
<code>prompt</code>	<code>str</code>	非空 UTF-8 提示词。
<code>input_type</code>	<code>str</code>	<code>prompt</code> （纯文本）或 <code>multimodal</code> （图文）。
<code>run_mode</code>	<code>str</code>	当前仅支持 <code>generate</code> 。
<code>retain_history</code>	<code>bool</code>	是否保留本轮 KV Cache；多轮对话需设为 <code>True</code> 。
<code>enable_think</code>	<code>bool</code>	是否启用模型思考模式。
<code>role</code>	<code>str</code>	Chat Template 使用的消息角色，不能为空。
<code>images</code>	<code>list / None</code>	VLM 图片列表；每项为 <code>{"data": bytes, "width": int, "height": int}</code> ， <code>data</code> 为 RGB888 紧密排列字节。
<code>img_start</code>	<code>str</code>	图片起始标记。
<code>img_end</code>	<code>str</code>	图片结束标记。
<code>img_content</code>	<code>str</code>	图片内容标记，须与 Prompt 中标记一致。
<code>user_data</code>	<code>object / None</code>	透传给 Token 回调的用户数据。

多模态运行必须在 `config()` 提供 `mmproj_path`，且 `images` 至少包含一张图片；图片数量、Prompt 中的 `img_content` 标记应匹配。

返回值：字典，包含 `text`（完整文本）、`token_ids`（生成 Token ID 列表）、`token_count`（生成数量）和 `status`（`RunStatus` 状态）。

4.3.6 reset_session

功能：清空当前会话的 KV Cache 和历史状态，不卸载模型；适合开始新对话。

返回值：无。

4.3.7 break_generation

功能：请求中断当前正在进行的生成。

返回值：无。生成线程收到信号后通过回调返回结束状态。

4.3.8 set_chat_template

功能：设置原生 Chat Template 的系统提示词及用户消息前后缀。

参数	类型	说明
<code>system_prompt</code>	<code>str</code>	系统消息内容。
<code>prompt_prefix</code>	<code>str</code>	每轮用户 Prompt 前追加的文本。
<code>prompt_postfix</code>	<code>str</code>	每轮用户 Prompt 后追加的文本。

返回值：无。

4.3.9 uninit

功能：停止 Runtime 并释放模型、KV Cache、NPU 上下文等资源。

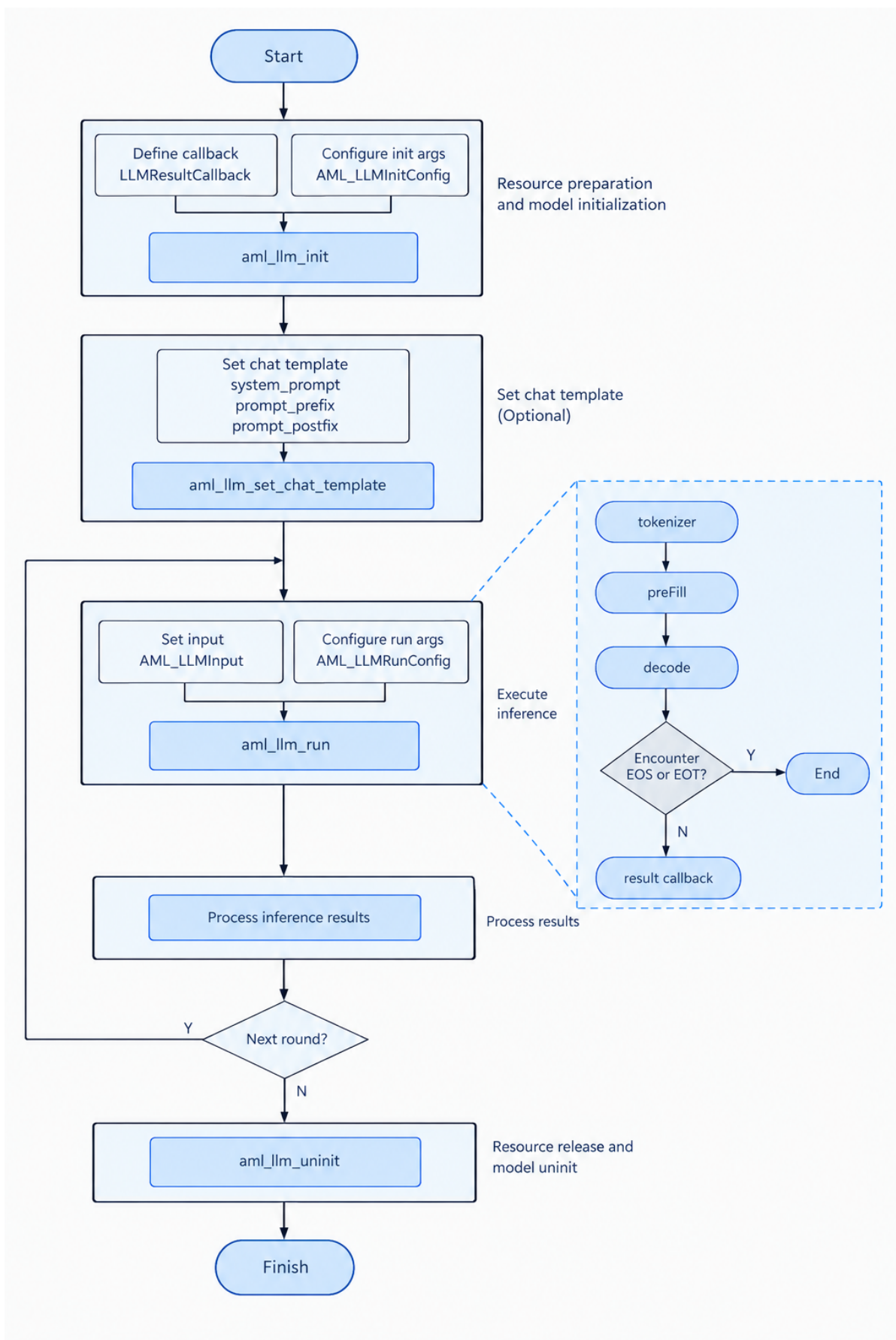
返回值：无。建议放在 `try...finally` 中调用；未初始化时会安全跳过。

5. 板端推理 (C++ API)

本章详细介绍 LLMSDK 的 C++ API 接口规范与调用流程。所有接口声明均位于头文件 `llmsdk.h` 中。为确保资源管理的正确性与推理的稳定性，开发者需严格遵循以下生命周期顺序：**定义回调与配置** → **初始化模型** → **构造输入并推理** → **会话管理（保留/重置）** → **释放上下文**。

注意：所有指针参数的有效性及生命周期管理，请严格参照各接口的具体说明。

5.1 流程简介



一、核心调用流程

1. **准备阶段:** 配置模型路径、采样参数及结果回调函数。
2. **初始化:** 调用 `aml_llm_init()` 创建运行时上下文。
3. **推理阶段:** 构造输入数据并提交推理请求。
4. **会话管理:** 根据业务需求保留历史上下文或重置会话。
5. **资源释放:** 调用 `aml_llm_uninit()` 清理所有资源。

关键约束：

- 上下文句柄（Context Handle）贯穿整个生命周期，需妥善保存。
- 严禁在初始化失败或资源释放后继续使用该句柄调用任何接口。
- 只有当 `aml_llm_init()` 成功返回后，方可进行推理操作。

二、模型类型与输入处理

SDK 支持纯文本 LLM 和多模态 VLM 两种场景，二者共用同一套运行接口，但在初始化配置与输入构造上存在差异。

1. 纯文本 LLM

- **输入方式：**直接传入 Prompt 文本。
- **处理机制：**SDK 内部结合 Chat Template 自动完成格式整理与分词（Tokenization），适用于常规对话场景。

2. 视觉语言模型 (VLM)

- **初始化配置：**除基础模型路径外，必须在扩展配置中指定 **Projector 模型路径**。
- **输入构造：**
 - **图像数据：**提交 RGB888 格式的原始图像数据。
 - **文本提示：**提交包含图像标记（Image Token）的 Prompt。
- **一致性要求：**
 - **数量一致：**提交的图像数量必须与 Prompt 中的图像标记数量相等。
 - **顺序一致：**图像数据的排列顺序必须与 Prompt 中标记出现的顺序严格对应。
 - **目的：**确保 SDK 能将视觉 Embedding 准确映射到文本序列的正确位置，实现图文对齐。

三、异步推理与回调机制

推理结果通过注册的 `LLMResultCallback` 回调函数异步返回，支持流式输出与状态监控。

- **回调触发：**每生成一个 Token 或推理状态发生变化时，SDK 将触发回调。
- **应用处理：**开发者可在回调中执行以下操作：
 - 累积生成的文本片段。
 - 实时更新 UI 界面以实现流式展示。
 - 触发后续业务逻辑（如中断生成、工具调用等）。

四、资源释放

当所有推理任务完成且回调处理结束后，**必须**调用 `aml_llm_uninit()` 接口以释放资源。

释放内容包括：

- 模型权重内存。
- KV Cache 缓存空间。
- 推理线程资源。
- NPU 硬件上下文句柄。

注意：建议在应用退出或模块卸载前显式调用此接口，避免内存泄漏或 NPU 资源占用导致后续加载失败。

5.2 基础数据结构和枚举

本节按 `llmsdk.h` 中的定义说明句柄、枚举和结构体。

5.2.1 LLMContext

类型 `LLMContext` 表示模型运行时句柄，实际类型为 `void*`。该句柄由 `aml_llm_init()` 输出，传给其他接口；`aml_llm_uninit()` 调用后句柄失效。

```
typedef void* LLMContext;
```

5.2.2 AML_LLMRunStatus

枚举 `AML_LLMRunStatus` 表示一次生成任务在回调中的运行状态。

回调状态枚举如下：

成员	说明
<code>AML_LLM_RUN_NORMAL</code>	推理进行中，本次回调携带一个生成结果。
<code>AML_LLM_RUN_FINISH</code>	推理完成，通常表示遇到 EOS 或达到生成上限。
<code>AML_LLM_RUN_ERROR</code>	推理发生错误。

5.2.3 AML_LLMRetStatus

枚举 `AML_LLMRetStatus` 表示 API 调用是否成功。

API 返回状态如下：

成员	说明
<code>AML_LLM_Status_Success</code>	调用成功。异步推理表示请求已接受。
<code>AML_LLM_Status_Failed</code>	调用失败，可能是句柄、参数、模型或设备资源无效。

5.2.4 AML_LLMInputType

枚举 `AML_LLMInputType` 表示 `AML_LLMInput` 使用的输入数据类型。

成员	说明
<code>AML_LLM_INPUT_PROMPT</code>	UTF-8 文本；SDK 自动应用 Chat Template 并分词。
<code>AML_LLM_INPUT_MULTIMODAL</code>	文本与 RGB888 图像/视频帧；SDK 执行视觉处理。
<code>AML_LLM_INPUT_TOKEN</code>	已完成模板和分词的 token ID，SDK 不再改写。

5.2.5 AML_LLMRunMode

枚举 `AML_LLMRunMode` 表示本次运行采用的推理模式。

成员	说明
----	----

成员	说明
<code>AML_LLM_RUN_GENERATE</code>	文本生成模式。
<code>AML_LLM_RUN_EMBEDDING</code>	向量生成模式，当前版本预留。

5.2.6 AML_LLMImageType

枚举 `AML_LLMImageType` 表示 VLM 图像数据的存储形式。

成员	说明
<code>AML_LLM_IMAGE_TYPE_BUFFER</code>	紧密排列的 RGB888 图像 buffer。

5.2.7 AML_LLMSamplingMode

枚举 `AML_LLMSamplingMode` 表示文本生成时采用的采样策略。

成员	说明
<code>AML_LLM_ARG_Max</code>	Argmax，选择概率最高 token，结果确定且可复现。
<code>AML_LLM_TOP_P</code>	Top-P 核采样； $0 < \text{top_p} \leq 1$ 。
<code>AML_LLM_TOP_K</code>	Top-K 采样； $\text{top_k} > 0$ 。
<code>AML_LLM_CHAIN_SAMPLER</code>	按 JSON sampler chain 配置采样，配置放在 <code>sampler_params_json</code> 。

5.2.8 AML_LLMInitConfig

结构体 `AML_LLMInitConfig` 表示创建 LLM/VLM 运行时上下文所需的初始化配置。

初始化配置结构体：

成员	类型	说明
<code>model_path</code>	<code>const char*</code>	语言模型 ADLA 文件路径。调用期间必须有效。
<code>sampling_mode</code>	<code>AML_LLMSamplingMode</code>	默认采样策略。
<code>top_k</code>	<code>int32_t</code>	Top-K 参数；使用 TOP_K 时必须大于 0。
<code>top_p</code>	<code>float</code>	Top-P 参数；使用 TOP_P 时取 $(0, 1]$ 。
<code>temperature</code>	<code>float</code>	温度系数，必须大于 0。
<code>repeat_penalty</code>	<code>float</code>	重复惩罚系数，必须大于 0；1.0 表示不惩罚。
<code>init_extend</code>	<code>AML_LLMInitExtend</code>	扩展参数。

5.2.9 AML_LLMInitExtend

结构体 `AML_LLMInitExtend` 表示初始化阶段的可选扩展参数。

成员	类型	说明
<code>mmproj_path</code>	<code>const char*</code>	VLM projector ADLA 路径；纯文本模型传 <code>NULL</code> 。
<code>sampler_params_json</code>	<code>const char*</code>	<code>AML_LLM_CHAIN_SAMPLER</code> 的 JSON 配置；不用时传 <code>NULL</code> 或空字符串。
<code>reserved</code>	<code>uint8_t[1024-...]</code>	保留字段，必须清零。

5.2.10 AML_LLMRunConfig

结构体 `AML_LLMRunConfig` 表示一次推理请求的运行模式、历史保留和运行时扩展配置。

成员	类型	说明
<code>run_mode</code>	<code>AML_LLMRunMode</code>	通常设置为 <code>AML_LLM_RUN_GENERATE</code> 。
<code>retain_history</code>	<code>int</code>	设为 1 保留 KV Cache 和对话/视觉历史；设为 0 不保留。
<code>enable_think</code>	<code>int</code>	设为 1 启用思考模式，是否生效取决于模型模板。
<code>run_extend</code>	<code>AML_LLMRunExtend</code>	扩展参数。

5.2.11 AML_LLMRunExtend

结构体 `AML_LLMRunExtend` 表示单次推理可覆盖的采样参数。

成员	类型	说明
<code>sampling_config_valid</code>	<code>uint32_t</code>	设为 1 使用本结构的全部采样字段；设为 0 使用初始化默认值。
<code>sampling_mode</code>	<code>AML_LLMSamplingMode</code>	本次采样策略。
<code>top_k</code> / <code>top_p</code>	<code>int32_t</code> / <code>float</code>	本次 Top-K/Top-P 参数。
<code>temperature</code> / <code>repeat_penalty</code>	<code>float</code> / <code>float</code>	本次温度和重复惩罚。
<code>sampler_params_json</code>	<code>const char*</code>	本次 Chain Sampler JSON；不用时传 <code>NULL</code> 。
<code>reserved</code>	<code>uint8_t[...]</code>	保留字段，必须清零。

5.2.12 AML_LLMImageInput

结构体 `AML_LLMImageInput` 表示一帧传给 VLM 的原始 RGB888 图像数据。

成员	类型	说明
<code>type</code>	<code>AML_LLMImageType</code>	当前仅支持 <code>AML_LLM_IMAGE_TYPE_BUFFER</code> 。
<code>data</code>	<code>const void*</code>	紧密排列的 RGB888 数据，大小为 <code>width*height*3</code> ，无 stride、行填充和文件头。
<code>width / height</code>	<code>uint32_t</code>	原始图像宽高，单位为像素。
<code>timestamp</code>	<code>uint64_t</code>	视频帧时间戳，单位毫秒；普通图片可填 0。

图像 buffer 必须在 `aml_llm_run()` 返回前保持有效；JPEG/PNG 等压缩数据需由应用先解码为 RGB888。

5.2.13 AML_LLMMultimodalInput

结构体 `AML_LLMMultimodalInput` 表示一次 VLM 请求中的文本提示词和图像数组。

成员	类型	说明
<code>prompt</code>	<code>const char*</code>	UTF-8 提示词，在图片位置放置 <code>img_content</code> 标记。
<code>img_inputs</code>	<code>const AML_LLMImageInput*</code>	图片/视频帧数组。顺序必须与 prompt 标记顺序一致。
<code>img_count</code>	<code>uint32_t</code>	数组元素个数；大于 0 时数组必须有效。
<code>img_start / img_end</code>	<code>const char*</code>	图片 embedding 前后的边界标记；FastVLM/MLP 通常为空。
<code>img_content</code>	<code>const char*</code>	prompt 中的图片占位符；传 NULL 时默认 <code><image></code> 。
<code>sample_rate_num / sample_rate_den</code>	<code>uint32_t</code>	视频采样率预留字段，当前 Runtime 不使用。

5.2.14 AML_LLMTokenInput

结构体 `AML_LLMTokenInput` 表示已经完成模板处理和分词的 token ID 输入。

成员	类型	说明
<code>input_ids</code>	<code>const int32_t*</code>	最终 token ID 数组；调用方负责模板、角色和分词。
<code>n_tokens</code>	<code>int32_t</code>	token 数量，必须大于 0。

5.2.15 AML_LLMInput

结构体 `AML_LLMInput` 表示传给 `aml_llm_run()` 的统一输入对象。

成员	类型	说明
<code>input_type</code>	<code>AML_LLMInputType</code>	决定使用联合体中的哪个成员。
<code>prompt_input</code>	<code>const char*</code>	<code>PROMPT</code> 类型文本。
<code>multimodal_input</code>	<code>AML_LLMMultimodalInput</code>	<code>MULTIMODAL</code> 类型输入。
<code>token_input</code>	<code>AML_LLMTokenInput</code>	<code>TOKEN</code> 类型输入。
<code>role</code>	<code>const char*</code>	消息角色，NULL 时默认为 <code>user</code> 。

5.2.16 AML_LLMGenerationResult

结构体 `AML_LLMGenerationResult` 表示文本生成模式下的一次增量输出，通常对应一个生成 token。

成员	类型	说明
<code>text</code>	<code>const char*</code>	当前 UTF-8 文本片段，仅在回调期间有效。
<code>token_id</code>	<code>int32_t</code>	当前生成 token 的 ID。

5.2.17 AML_LLMEmbeddingResult

结构体 `AML_LLMEmbeddingResult` 表示向量模式下返回的 embedding 数据；该模式在当前版本中预留。

成员	类型	说明
<code>values</code>	<code>const float*</code>	embedding 浮点数组，仅在回调期间有效。
<code>dimension</code>	<code>uint32_t</code>	<code>values</code> 中的浮点元素数量。

5.2.18 AML_LLMResult

结构体 `AML_LLMResult` 表示一次推理回调返回的结果。实际使用时，根据 `AML_LLMRunConfig.run_mode` 选择 `generation` 或 `embedding` 成员；其中指针指向的内容只在回调期间有效。

成员	类型	说明
<code>generation.text</code>	<code>const char*</code>	当前 UTF-8 文本片段，仅在回调期间有效。
<code>generation.token_id</code>	<code>int32_t</code>	当前 token ID。
<code>embedding.values</code>	<code>const float*</code>	embedding 数据，当前为预留功能。
<code>embedding.dimension</code>	<code>uint32_t</code>	embedding 维度。

5.3 基础 API 介绍

本节介绍结果回调和模型初始化、推理、资源释放等基础 API。回调函数由应用注册，SDK 在生成过程中按 token 或状态变化调用。

5.3.1 LLMResultCallback

函数类型 `LLMResultCallback` 定义推理结果回调的参数和调用约定。SDK 通过 `result` 返回结果，通过 `userdata` 原样传递应用上下文，并通过 `state` 通知本次推理进行中、完成或出错。

```
typedef void (*LLMResultCallback)(AML_LLMResult *result, void *userdata,
                                  AML_LLMRunStatus state);
```

参数	类型	说明
<code>result</code>	<code>AML_LLMResult*</code>	当前生成结果；状态为错误时可能为 <code>NULL</code> 。
<code>userdata</code>	<code>void*</code>	应用传入的用户数据指针。
<code>state</code>	<code>AML_LLMRunStatus</code>	本次回调的运行状态。

回调中应快速处理数据；如需异步使用 `text` 或 `values`，必须复制内容。

5.3.2 aml_llm_init

接口 `aml_llm_init` 用于加载语言模型和可选的 VLM projector、创建运行时上下文并注册结果回调。

API	<code>aml_llm_init</code>
功能	加载语言模型（以及可选的 VLM projector），创建 <code>LLMContext</code> ，注册结果回调。
参数	<code>LLMContext* context</code> ：输出上下文句柄。 <code>AML_LLInitConfig* init_config</code> ：初始化配置。 <code>LLMResultCallback callback</code> ：流式结果回调，可传 <code>NULL</code> 。
返回值	<code>AML_LLM_Status_Success</code> ：初始化成功。 <code>AML_LLM_Status_Failed</code> ：模型、参数、设备或内存错误。

示例：

```
LLMContext context = NULL;
AML_LLInitConfig cfg = {0};
cfg.model_path = "model.ad1a";
cfg.sampling_mode = AML_LLM_ARG_Max;
cfg.top_k = 3; cfg.top_p = 0.9f;
cfg.temperature = 1.0f; cfg.repeat_penalty = 1.1f;
AML_LLMRetStatus ret = aml_llm_init(&context, &cfg, callback);
```

5.3.3 aml_llm_run

接口 `aml_llm_run` 用于提交一次文本、Token 或多模态推理请求，生成结果通过初始化时注册的回调按 token 返回。

API	<code>aml_llm_run</code>
功能	执行一次推理并异步触发回调； <code>retain_history=1</code> 时保留会话 KV Cache。
参数	<code>LLMContext context</code> ：有效上下文。 <code>AML_LLMInput* input</code> ：本次输入。 <code>AML_LLMRunConfig* run_config</code> ：运行模式、历史和采样覆盖配置。 <code>void* userdata</code> ：回调用户数据，SDK 原样传回。
返回值	<code>AML_LLM_Status_Success</code> ：请求已接受。 <code>AML_LLM_Status_Failed</code> ：上下文、输入或运行配置非法。

示例：

```
AML_LLMInput input = {0};
input.input_type = AML_LLM_INPUT_PROMPT;
input.prompt_input = "你好，请介绍一下自己。";
AML_LLMRunConfig run = {0};
run.run_mode = AML_LLM_RUN_GENERATE;
run.retain_history = 1;
AML_LLMRetStatus ret = aml_llm_run(context, &input, &run, userdata);
```

5.3.4 aml_llm_uninit

接口 `aml_llm_uninit` 用于释放模型、KV Cache、线程和上下文资源。

API	<code>aml_llm_uninit</code>
功能	卸载模型并销毁上下文。调用后不得再次使用该句柄。
参数	<code>LLMContext context</code> ： <code>aml_llm_init()</code> 返回的有效句柄；应在所有回调结束后调用。
返回值	<code>AML_LLM_Status_Success</code> ：释放成功。 <code>AML_LLM_Status_Failed</code> ：句柄无效或释放失败。

示例：

```
aml_llm_uninit(context);
context = NULL;
```

5.3.5 aml_llm_reset

接口 `aml_llm_reset` 用于清空当前会话的 KV Cache、文本历史和视觉 embedding，使上下文恢复到可开始新会话的状态。

API	aml_llm_reset
功能	清空 KV Cache、文本历史和视觉 embedding，使上下文开始新会话。
参数	LLMContext context：有效上下文；运行中应先调用 aml_llm_break() 并等待回调结束。
返回值	AML_LLM_Status_Success：重置成功。 AML_LLM_Status_Failed：句柄无效或重置失败。

示例：

```
AML_LLMRetStatus ret = aml_llm_reset(context);
```

5.3.6 aml_llm_break

接口 aml_llm_break 用于请求中断正在执行的生成任务；任务最终状态仍通过 LLMResultCallback 通知。

API	aml_llm_break
功能	请求中断当前生成任务；最终状态仍由回调通知。
参数	LLMContext context：正在推理的有效上下文。
返回值	AML_LLM_Status_Success：中断请求已提交。 AML_LLM_Status_Failed：句柄无效或提交失败。

示例：

```
aml_llm_break(context);
```

5.3.7 aml_llm_set_chat_template

接口 aml_llm_set_chat_template 用于设置后续 Prompt 输入使用的 system prompt、用户输入前缀和后缀模板。

API	aml_llm_set_chat_template
功能	设置 system prompt、用户输入前缀和后缀，覆盖后续 Prompt 请求的模板片段。Token 输入不经过该模板。
参数	LLMContext context：有效上下文。 const char* system_prompt：系统提示词，可为 NULL。 const char* prompt_prefix：用户输入前缀。 const char* prompt_postfix：用户输入后缀。
返回值	AML_LLM_Status_Success：设置成功。 AML_LLM_Status_Failed：句柄或字符串参数无效。

示例：

```
aml_llm_set_chat_template(context,
                        "<|im_start|>system\nYou are a helpful assistant.",
                        "<|im_end|>\n",
                        "<|im_start|>user\n",
                        "<|im_end|>\n<|im_start|>assistant\n");
```

5.4 错误返回码

LLMSDK 当前 API 使用统一的二值返回码：

返回值	说明
AML_LLM_Status_Success	接口调用成功；异步推理表示请求已提交。
AML_LLM_Status_Failed	接口调用失败。具体原因通过日志和参数检查定位，包括上下文无效、模型加载失败、输入非法、采样参数越界、内存不足或设备不可用。

5.5 多模态说明

LLMSDK 通过 AML_LLM_INPUT_MULTIMODAL 支持 VLM 图文推理。语言模型和 projector 必须分别转换为 ADLA 文件，并在 AML_LLMInitExtend.mmproj_path 指定 projector。SDK 负责图像缩放、归一化、视觉编码和 embedding 注入；应用负责图像解码和 buffer 管理。

5.5.1 多模态调用流程

- 在 AML_LLMInitConfig 中设置语言模型路径和 init_extend.mmproj_path。
- 将压缩图片解码为紧密排列的 RGB888 buffer。
- 构造 AML_LLMImageInput 和 AML_LLMMultimodalInput，使图片数组顺序与 prompt 中 <image> 标记一致。
- 设置 input.input_type = AML_LLM_INPUT_MULTIMODAL，调用 aml_llm_run()。
- 首轮将 retain_history 设为 1，后续纯文本请求可复用图像 embedding 和 KV Cache。
- 新会话调用 aml_llm_reset()；结束时调用 aml_llm_uninit()。

5.5.2 VLM 示例

```
AML_LLMInitConfig cfg = {0};
cfg.model_path = "fastvlm.adla";
cfg.sampling_mode = AML_LLM_ARG_Max;
cfg.top_k = 3; cfg.top_p = 0.9f;
cfg.temperature = 1.0f; cfg.repeat_penalty = 1.1f;
cfg.init_extend.mmproj_path = "mmproj.adla";
LLMContext ctx = NULL;
aml_llm_init(&ctx, &cfg, callback);

AML_LLMImageInput image = {0};
image.type = AML_LLM_IMAGE_TYPE_BUFFER;
image.data = rgb888_data;
image.width = width; image.height = height;
AML_LLMInput input = {0};
input.input_type = AML_LLM_INPUT_MULTIMODAL;
input.role = "user";
```

```
input.multimodal_input.prompt = "<image>Describe this image";
input.multimodal_input.img_inputs = &image;
input.multimodal_input.img_count = 1;
input.multimodal_input.img_content = "<image>";
AML_LLMRunConfig run = {0};
run.run_mode = AML_LLM_RUN_GENERATE;
run.retain_history = 1;
aml_llm_run(ctx, &input, &run, NULL);

/* 保留历史时，后续可发送纯文本问题 */
AML_LLMInput followup = {0};
followup.input_type = AML_LLM_INPUT_PROMPT;
followup.prompt_input = "还有其他内容吗? ";
aml_llm_run(ctx, &followup, &run, NULL);
```

视频场景由应用使用 FFmpeg 等组件抽帧后按时间顺序传入；`timestamp` 使用毫秒。图像数据和字符串必须保持有效到 `aml_llm_run()` 返回。

6. 常见问题与性能参考

6.1 性能参考数据

有关 LLMSDK 在不同模型和硬件平台上的详细性能测试数据，包括吞吐量（Tokens/秒）和延迟等指标，请参考以下 GitHub 链接：

性能参考数据：Amlogic LLMSDK 性能参考数据

6.2 常见错误排查

问题 1：是否能支持客户自己的模型？

解答：如果客户模型是基于第一章 Model_list 里的模型 fine-tuning 得到的模型，当前是可以支持转换部署的。其他LLM暂时不支持，更多的LLM模型，在后续版本中Release。

6.3 参考链接

https://github.com/Amlogic-NN/amlInn-toolkit/tree/main/amlInn_toolkit_lite/amlInn_edge_toolkit_lite

<https://github.com/Amlogic-NN/amlInn-model-playground/tree/main/examples/LLMs>